Taller Lenguaje ensamblador

1. Implemente en assembler Hack las siguientes tareas:

Donde esta cada variable en la ram

Nombre	Dirección
j	3012
sum	4500
q	3812
arr	20561
sum	2200
x	4000
i	6151
END	20
NEXT	16

Tabla 1. Tabla de direcciones

a. Sum = 0

// RAM[2200] = 0

@ 2200

M = 0

M = 0

A 2200 D

b. J = J + 1 // M = M + 1

@ 3012

M = M + 1

M = M + 1

A 3012

c. q = sum + 12 - j

// RAM [3812] = RAM [4500] + 12 - RAM [3012]

@ 4500

D = M

@ 12

D = D + A

@ 3012

D = D - M

@ 3812

M = D

4500 M = ?

M = D + A

M = D - M

M = D

A 4500

12

3012

3812

D ?

D + A

D - M

d. arr [3] = -1

Si el arr empieza en la dirección 20561

Entonces arr[0] → RAM [20561]

arr [1] → RAM [20562]

arr [2] → RAM [20563]

arr [3] → RAM [20564]

arr [i] → RAM [Base + i]

// RAM [20564] = -1

@ 3

D = A

@ 20561

A = A + D

M = -1

M = -1

A 3

20561

20561 + D

D 3

e. arr [j] = 0

// arr [3012] = 0

20561

arr [i] → arr [base + i]

@ 3012

D = A

D = M

@ 20561

A = A + D

M = 0

A 3012

20561

20561 + D

M = ?

D = j ?

D 3012

20561

f. arr [j] = 17

j + arr

// arr [3012] = 17

RAM [RAM [3012], 20

@ 3012

D = M

@ 20561

D = A + D

@ 16 guarda Temp

M = D

@ 17

D = A

M = RAM [3012]

M

M = D

A 3012

20561

16

17

valor j

D RAM [3012]

20561 + RAM [3012]

A + D

17

vea la instrucción en la dirección 50

g. goto 50

A = 50

@ 50

comp 0 → No afecta, permite el salto

dest null

Jump JMP → Salta siempre

0 ; JMP

h. if D == 0 goto 112

@ 112

comp D

dest null

Jump JEQ

D ; JEQ

i. if D<9 goto 507

D-9

@ 9

D=D-A

A 9

D D-A

@ 507

D; JLT

comp D

dest null

Jump JLT

j. if RAM[12]>0 goto 50

Ram [12]

M=?

@ 12

D=M

A 12

D ?

@ 50

D; JGT

comp D

dest null

Jump JGT

k. if sum>0 goto End

RAM[2200]

Ram [20]

M= RAM[2200]

@ 2200

D=M

A 2200

20

D M

@ 20

D; JGT

comp D

Dest null

Jump JGT

l. if x[i]≤0 goto next

[i] → arr + i

i

x [RAM [6151]]

→ RAM [4000]

M= RAM [6151]

M= Ram [4000+ i]

@ 6151

D=M

A 6151

x[i]

4000 + D

16

D M

M

@ 4000

A= A+D

D=M

@ 16

D; JLE

comp D

Dest null

Jump JLE

2. Traduzca de lenguaje ensamblador a binario las siguientes instrucciones.

a. @sum A

Tipo A

14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0 0 0 1 0 00 1 1 00 1 0 100

@ 4500 16 bit

4500 |2
0 2250 |2
0 1125 |2
1 562 |2
0 281 |2
1 140 |2
0 70 |2
0 35 |2
1 17 |2
1 8 |2
0 4 |2
0 2 |2
0 1

1 0 00 1 1 00 1 0 100

b. M=0 Tipo C

dest

comp 0
dest M
Jum null

111 a cccccc ddd jjj
0 101010 001 000

c. @0 Tipo A

0 000000000000000

d. D=M
Dest comp

Dest D
comp M
Jump Null

1 1 1 1 110000 010 000

e. @i Tipo A

@ 6151

6151 |2
1 3075 |2
1 1537 |2
1 768 |2
0 384 |2
0 192 |2
0 96 |2
0 48 |2
0 24 |2
0 12 |2
0 6 |2
0 3 |2
1 1

0 0 1 1 00000000 111

f. M=D

Dest M 111 0 001100 001 000
comp D
Jump null

g. @i 0 0 1 1 00000000 111

h. MD = M-1

Dest MD 111 1 110010 011 000
comp M-1
Jump null

i. @ Next
@ 16

16 |2
0 8 |2
0 4 |2
0 2 |2
0 1

1000

0 0000000000 1000

j. D; JLT

Dest 111 0 001100 000 100
comp
Jump JLT

3. En que cosiste realizar el desamblado de un programa?

Traducir a instrucciones tipo a y tipo c.

4. Realizalice el desensamble de los siguientes números Binarios.

Tipo A	0000000000010000	2^4	@ 16
Tipo C	1110101010001000		M = 0
Tipo A	0000000000000000		@ 0
Tipo C	1111110000010000		D = M
Tipo A	0000000000010001	$2^4 + 2^0$	@ 17
Tipo C	1110001100001000		M = 0
Tipo A	0000000000010001	$2^4 + 2^0$	@ 17
Tipo C	1111110010011000		DM = M-1
Tipo A	0000000000010000	2^4	@ 16
Tipo C	1110001100000100		D ; JLT
Tipo A	0000000000000001	2^0	@ 1
Tipo C	1111110000010000		D = M
Tipo A	0000000000010000	2^4	@ 16
Tipo C	1111000010001000		M = D + M
Tipo A	0000000000000110	$2^1 + 2^2$	@ 6
Tipo C	1110101010000111		null = 0 ; JMP
Tipo A	0000000000010000	2^4	@ 16
Tipo C	1111110000010000		D = M
Tipo A	0000000000000010	2^1	@ 2
Tipo C	1110001100001000		M = D
Tipo A	0000000000010100	$2^4 + 2^2$	@ 20
Tipo C	1110101010000111		null = 0 ; JMP

comp Dest

Dest = comp ; Jump

comp		c	c	c	c	c	c
0		1	0	1	0	1	0
1		1	1	1	1	1	1
-1		1	1	1	0	1	0
D		0	0	1	1	0	0
A	M	1	1	0	0	0	0
!D		0	0	1	1	0	1
!A	!M	1	1	0	0	0	1
-D		0	0	1	1	1	1
-A	-M	1	1	0	0	1	1
D+1		0	1	1	1	1	1
A+1	M+1	1	1	0	1	1	1
D-1		0	0	1	1	1	0
A-1	M-1	1	1	0	0	1	0
D+A	D+M	0	0	0	0	1	0
D-A	D-M	0	1	0	0	1	1
A-D	M-D	0	0	0	1	1	1
D&A	D&M	0	0	0	0	0	0
D|A	D|M	0	1	0	1	0	1
a==0	a==1						

dest	d	d	d	acción: el valor se almacena en:
null	0	0	0	El valor no se almacena
M	0	0	1	RAM[A]
D	0	1	0	Registro D
DM	0	1	1	Registro D y RAM[A]
A	1	0	0	Registro A
AM	1	0	1	Registro A y RAM[A]
AD	1	1	0	Registro A y Registro D
ADM	1	1	1	Registro A, Registro D y RAM[A]

jump	j	j	j	efecto:
null	0	0	0	No salta
JGT	0	0	1	Si comp > 0 salta
JEQ	0	1	0	Si comp = 0 salta
JGE	0	1	1	Si comp ≥ 0 salta
JLT	1	0	0	Si comp < 0 salta
JNE	1	0	1	Si comp ≠ 0 salta
JLE	1	1	0	Si comp ≤ 0 salta
JMP	1	1	1	Salto incondicional

El programa multiplica dos números mediante sumas sucesivas.